

KOL Scraper Automation

Handover & Technical Reference Documentation

Project	kol-scraper v1.0.0
Description	Auto-scrapes TikTok post metrics from a Google Spreadsheet via Apify, then writes the results back to the sheet.
Stack	Node.js 22 • TypeScript (ESM) • Apify REST API • Google Sheets API v4
Document type	Technical handover
Date created	13 July 2026
Prepared and made by	Guntur Riarto Adhyatma
Contact	0881080301088

This document describes the architecture, logic flow, configuration, how to run, and important notes (including potential issues) of the KOL Scraper project. It is intended for whoever will take over and operate this project after the handover.

1. Project Overview

KOL Scraper is an automated service that reads a list of TikTok post links from a Google Spreadsheet (the "KOL Mastersheet"), scrapes each post's metrics through the Apify TikTok Scraper, then writes the resulting metrics (impression, view, likes, share, comment, save, timestamp) back into the same row of the sheet.

The service can run once (**--once** mode, for testing/manual runs) or as a **cron job** that runs automatically on a schedule.

Flow at a glance

```
Google Sheet (14 tabs)
  | read column H (Link Post), filter by date (column O, last 30 days)
  v
Batching --> split into groups of <= BATCH_SIZE URLs
  v
Apify TikTok Scraper (clockworks-tiktok-scraper) --> poll until SUCCEEDED
  v
Mapper --> match results back to rows via URL / video ID
  v
Google Sheet --> write columns I..O per row
```

Data scope

- **14 sheets**: 2 platforms [TT] & [YC] × 7 categories.
- 7 categories: Bodycare, Skincare, Mattedorable, Next Level, NCO, FYNE, Eomma.
- [IG] sheets are skipped (not processed).
- Only valid TikTok URLs are scraped (tiktok.com, vt.tiktok.com, vm.tiktok.com).

2. Technology & Dependencies

Package	Version	Purpose
googleapis	^140.0.0	Service-account auth + read/write Google Sheets API v4
apify-client	^2.9.0	Installed as a dependency, BUT the actual code uses raw fetch to the REST API (see notes)
node-cron	^3.0.3	Scheduler for cron mode
dotenv	^16.4.5	Loads variables from the .env file
zod	^3.23.8	Environment schema validation & parsing
pino	^9.1.0	Structured logging to stdout
typescript / tsx	^5.5.3 / ^4.16.2	Compiler + dev runner (dev tooling)

Runtime: **Node.js 22.x** (compile target ES2022, ESM modules — "type": "module"). Practical minimum Node 18+, recommended 20/22.

3. Prerequisites to Run

Mandatory checklist before running:

#	Item	Notes
1	Node.js 18+ (20/22 recommended) & npm	Check: <code>node -v</code>
2	Dependencies installed	Run <code>npm install</code>
3	.env file	Contains SPREADSHEET_ID & APIFY_API_TOKEN (see section 4)
4	Google Service Account JSON	Stored at <code>keys/service-account.json</code>
5	Spreadsheet shared with the service account	Role Editor , using the <code>client_email</code> from the JSON
6	Google Sheets API enabled	Enable it in Google Cloud Console for the service account's project
7	Apify account + API token	Actor <code>clockworks/tiktok-scraper</code> (paid per usage)
8	Sheet tabs named correctly	14 tabs: [TT] <Category> & [YC] <Category>

Google Service Account setup (brief)

- Open Google Cloud Console → create/select a project.
- Enable **Google Sheets API**.
- IAM & Admin → Service Accounts → create a service account → download the JSON key.
- Save the JSON to `keys/service-account.json`.
- Open the target spreadsheet → Share → add the `client_email` from the JSON → set to **Editor**.

4. Environment Configuration (.env)

All configuration is validated by zod in `src/config.ts`. If a required variable is empty, the app crashes immediately on start (fail-fast).

Variable	Required?	Default	Notes
SPREADSHEET_ID	Yes	—	Spreadsheet ID (from the sheet URL)
APIFY_API_TOKEN	Yes	—	Apify API token
GOOGLE_SERVICE_ACCOUNT_PATH	No	<code>./keys/service-account.json</code>	Path to the JSON key file
BATCH_SIZE	No	400	Number of URLs per batch (max 500)
APIFY_POLL_INTERVAL_MS	No	5000	Run-status polling interval (ms)
APIFY_POLL_TIMEOUT_MS	No	600000	Polling timeout per batch (10 minutes)
LOG_LEVEL	No	info	<code>trace debug info warn error fatal</code>

Example .env contents:

```
SPREADSHEET_ID=1AbC...spreadsheet_id_here
APIFY_API_TOKEN=apify_api_XXXXXXXXXXXXXXXXXX
GOOGLE_SERVICE_ACCOUNT_PATH=./keys/service-account.json
BATCH_SIZE=400
LOG_LEVEL=info
```

■ Credentials & security

The `.env` file and the `keys/` folder contain secrets and are already in `.gitignore` — **do not** commit them or send them over insecure channels. During handover, transfer both files separately and securely. Rotate the Apify token and re-generate the service-account key if needed.

5. How to Run

```
# 1. Install dependencies
npm install

# 2. Prepare configuration
cp .env.example .env      # then edit .env
#   make sure keys/service-account.json exists

# 3a. Run once (test/manual) -- uses tsx, no build
npm run dev               # = tsx src/index.ts --once

# 3b. Build then run once from the compiled output
npm run build              # tsc -> dist/ folder
npm run start:once         # node dist/index.js --once

# 3c. Cron mode (runs continuously in background/on a server)
npm run build && npm start # node dist/index.js
```

Script	Command	Purpose
dev	tsx src/index.ts --once	Run once directly from TS (development/test)
build	tsc	Compile TS → dist/
start:once	node dist/index.js --once	Run once from the build output
start	node dist/index.js	Cron mode (daemon, runs continuously)

For production, cron mode should run under a process manager (e.g. **pm2**, **systemd**, or **screen/tmux**) so it stays alive and auto-restarts. Example: `pm2 start dist/index.js --name kol-scraper`.

6. File Structure & Responsibilities

File	Responsibility
src/index.ts	Entry point. Detects --once vs cron flag. Registers the cron job. Handles unhandledRejection / uncaughtException.
src/config.ts	Env validation (zod). Defines CATEGORIES, PLATFORMS_TO_SCRAPE, & getTargetSheetNames() (the 14 sheet names).

File	Responsibility
src/scrapper/orchestrator.ts	The brain of the flow: read → batch → scrape → map → write. Builds the ScrapeSummary.
src/scrapper/mapper.ts	Matches Apify results to sheet rows (via URL / video ID) → assembles column I..O values.
src/sheets/client.ts	GoogleAuth service-account auth (spreadsheets scope). Singleton client.
src/sheets/reader.ts	Reads the 14 sheets, picks column H (link), filters by 30-day date (column O). Multi-format date parsing.
src/sheets/writer.ts	Batch-updates columns I:O per row via values.batchUpdate.
src/apify/client.ts	Calls the Apify REST API: start run, poll status, fetch dataset items, normalize results.
src/apify/batching.ts	Filters valid TikTok URLs + splits them into batches by BATCH_SIZE.
src/utis/logger.ts	The pino logger instance (stdout).
src/utis/date.ts	getWIBTimestamp() — Asia/Jakarta timestamp, format YYYY-MM-DD HH:mm:ss.

7. Per-Module Logic Detail

7.1 Entry & Scheduling (index.ts)

- If the arguments include `--once` → run `runScrapeCycle()` once. Exit code 1 if there are errors, 0 if clean.
- If no flag → register a cron with expression `0 16 */3 * *`, timezone `Asia/Jakarta`.
- The process stays alive waiting for the next cron trigger.

■ Schedule mismatch (needs clarification)

The code comments & README state a schedule of **23:00 WIB** (assuming 16:00 UTC). However the cron is set to `0 16 */3 * *` with **timezone Asia/Jakarta**, so it actually triggers at **16:00 WIB** every 3 days, **not** 23:00 WIB. If 23:00 WIB is intended, change it to `0 23 */3 * *` (keep timezone Asia/Jakarta). Confirm which one is correct before operating it.

7.2 Orchestrator (orchestrator.ts)

Steps in `runScrapeCycle()`:

- Initialize the Google Sheets client.
- `readAllLinks()` → collect all valid links from the 14 sheets.
- `createBatches()` → split into batches. If 0 links/batches, exit early.
- Loop over each batch **sequentially**: `runTikTokBatch()`, with a 2-second pause between batches. Per-batch errors are caught & recorded, and the cycle continues.
- `mapResultsToUpdates()` merges all results → `writeResults()` writes to the sheets.
- Returns a `ScrapeSummary` (`sheetsProcessed`, `totalUrls`, `batchesCreated/Succeeded/Failed`, `resultsWritten`, `errors[]`).

7.3 Reader (sheets/reader.ts)

- Reads the range `<sheet>!A:O` with `FORMATTED_VALUE`.
- Skips row 1 (the "KOL Mastersheet" header).
- Takes **column H** (index 7) as the **Link Post**; rows without a link are skipped.
- Takes **column O** (index 14) as the **Date Posting** for filtering: only posts from the **last 30 days**.
- The date parser supports 3 formats: `M/D/YYYY`, `DD-Mon-YYYY`, `DD Mon YYYY`. Years outside 2025–2027 or unparseable → skipped.
- Per-sheet read errors are caught → return an empty array for that sheet (does not fail the other sheets).

7.4 Batching (apify/batching.ts)

- Keeps only hosts `tiktok.com` / `vt.tiktok.com` / `vm.tiktok.com`; the rest are dropped & logged.
- Splits into chunks of `BATCH_SIZE` (default 400).
- Each batch keeps `urls[]` + `LinkRefs[]` (a back-reference to sheet+row for mapping).

7.5 Apify Client (apify/client.ts)

- POST to `/v2/acts/clockworks~tiktok-scraper/runs` with input: `postURLs`, `resultsPerPage=1`, `no comments/videos/covers`, `proxyCountryCode=None`.

- Poll `/v2/actor-runs/<runId>` every 5s until status SUCCEEDED (or FAILED/ABORTED/TIMED-OUT → throw).
- Total timeout per batch is 10 minutes (`APIFY_POLL_TIMEOUT_MS`).
- Fetch dataset items → map to: url, playCount, diggCount, shareCount, commentCount, collectCount (default 0 if empty).
- URLs are cleaned (`cleanUrl`): strip query, hash, trailing slash.

7.6 Mapper (scraper/mapper.ts)

- Builds a result lookup keyed by URL, then matches each link: **direct match** (normalized URL) → fallback **video-ID match** (pattern `/video/<id>`).
- If no match is found: the row is still written with value **0** + timestamp (see the risk note).
- Assembles column I..O values per row.

7.7 Writer (sheets/writer.ts)

- Groups updates per sheet, sorts by row number.
- `values.batchUpdate` with `valueInputOption: USER_ENTERED`, per-row range `<sheet>!I<row>:O<row>`.
- Per-sheet write errors are caught & logged (does not stop the other sheets).

8. Spreadsheet Column Mapping

Columns read as input:

Column	Index (0-based)	Name	Role
H	7	Link Post	The TikTok URL to scrape
O	14	Date Posting	Used to filter to the last 30 days

Columns written as output (I through O):

Column	Field	Apify source
I	Impression	playCount
J	View	playCount (same as I)
K	Likes	diggCount
L	Share	shareCount
M	Comment	commentCount
N	Save	collectCount
O	Update timestamp	getWIBTimestamp() (Asia/Jakarta)

■ Column O conflict (MUST be checked against the real sheet layout)

Column **O** is read as the **Date Posting** (30-day filter) in the reader, BUT it is also **written** as the **update timestamp** by the writer. This means after the first run, column **O** may become the scrape timestamp, so the date filter on the next run reads the scrape timestamp — not the real posting date. Likely causes: (a) the actual sheet layout has more/different columns, or (b) it is genuinely a mapping bug. **Verify the column order in the real spreadsheet** before running in production.

9. Important Notes & Potential Issues

The following points should be verified/fixed by whoever takes over the project:

#	Issue	Impact / Recommendation
1	Cron schedule 16:00 vs 23:00 WIB	Comments/README say 23:00 WIB but the code triggers at 16:00 WIB. Confirm & adjust the expression.
2	Column O has a dual role (Date Posting & Timestamp)	Can break the date filter on the next run. Check the real sheet column layout.
3	Unmatched URLs are written as 0	If Apify fails/misses a URL, old metrics get overwritten with 0. Consider skipping the write when unmatched.
4	BATCH_SIZE default 400 vs comment '50/batch'	The orchestrator/README comments mention 50; the effective value is 400. Align the docs/value.
5	apify-client installed but unused	The code uses raw fetch. It can be removed from dependencies, or migrated to the official SDK.
6	Apify cost	The actor is paid per usage. Large batches + scheduled runs = cost. Monitor the Apify quota/credit.
7	No retry on failed batches	A FAILED batch is only recorded in summary.errors; it is not retried automatically.
8	No automated tests	There are no unit/integration tests yet. Logic changes risk regressions.

10. Quick Troubleshooting

Symptom	Likely cause & fix
Crash on start, zod / env error	SPREADSHEET_ID or APIFY_API_TOKEN is empty in .env. Fill both in.
403 / permission error when reading-writing the sheet	The spreadsheet hasn't been shared with the service account's client_email as Editor, or the Sheets API isn't enabled.
'No URLs found to scrape'	All rows were filtered out (date >30 days / unrecognized date format / empty column H). Check columns H & O.
Apify run FAILED / TIMED-OUT	Wrong token/quota exhausted, invalid URLs, or batch too large. Check the Apify dashboard & lower BATCH_SIZE.
Many rows become 0	URLs did not match the scrape results (broken URL / vt/vm redirect). Check the 'No match found for URL' log.
Sheet name not found	Tabs must be exactly '[TT] <Category>' / '[YC] <Category>'. Check spelling & the spaces in the category.

All activity is logged via pino to stdout. For detailed debugging, set LOG_LEVEL=debug in .env to see per-sheet details (row counts, skippedByDate, etc.).

Prepared and made by Guntur Riarto Adhyatma — 0881080301088.

This handover document was generated from a direct reading of the project's source code (kol-scraper v1.0.0) as of 13 July 2026. For implementation details, refer directly to the files in the src/ folder.